



Distributed Systems

Fault Tolerance & Recovery

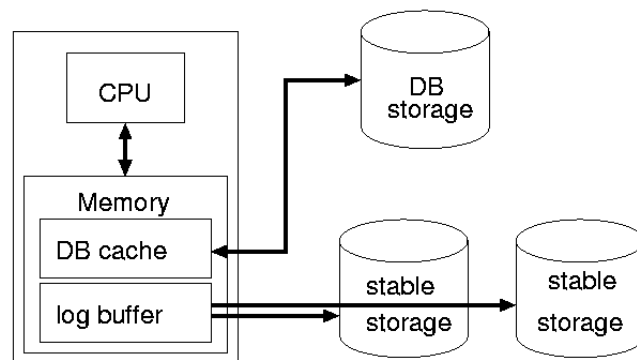


Recap



- Lamport et. al. solution for Byzantine agreement problem
- Iterative solution for Byzantine agreement problem
- Error recovery
 - Forward-error Recovery
 - Backward-error Recovery
 - Operation based approach
 - State-based approach

System Model



Some Issues in Crash Recovery

- *disk ^{HDD} writes are only atomic by sector*
- *updates and commits require multiple writes*
- *a crash may occur between writes*
- *log contains record of updates, commits, and aborts*
- *data is written to disk asynchronously*
 - *DB is cached*
 - *log is buffered*

Basic Deferred-Update Model

- save a transaction's updates as it runs, in temporary storage
- **update:** record a redo record (e.g. the new value of the item being updated) in an intention list
- use the saved updates to update the database when the transaction commits
- **commit:** update the database by applying the intention list in order, starting with the first operation
- **abort:** discard the transaction's intention list
- **read?**

ACID

Update-In-Place Model

- update the DB as transaction runs
- undo the updates if the transaction aborts

T1
 r
 (V)
 r
 (V)
 ...
 abort

Operation based approach (Updating-in-place)

- For every update a **log** is recorded in stable storage

1. Object name (or ID)
2. Old state of the object (used for UNDO)
3. New state of the object (used for REDO)

A **recoverable update operation** can be implemented as a **collection of operations**

1. do
2. undo
3. redo
4. display

Crash Recovery with Update-In-Place

To **reconstruct the DB system in event of a crash**, starting from an archived snapshot and the subsequent log:

- transactions not logged as committed are treated as aborted
- back out active or aborted transactions, using undo records
- do DB updates that may have been in cache, using redo records

Operation based approach

- **Implementing do operation:** A do operation can't be undone if the system crashes after an update is written on disk but before the log is stored.
- **Write-ahead log Protocol**
 - Update an object only after the undo log is recorded
 - Before committing the update, redo and undo logs are recorded
- Writing a log record for every update is expensive especially if failures are rare
- Use state based approach?

Atomic Actions and Commit Protocols

An action may have multiple sub-actions executed by different processes at different nodes of a distributed system

Atomic action : either all sub-actions are done or none are done (all-or-nothing property/ **global atomicity property**)

Atomic actions are basic building blocks in constructing fault tolerant operations.

Process P1

-

-

Lock (X)

X = X + Z

Unlock (X)

—

—

failure

Process P2

-

-

Lock (X)

X = X + Y

Unlock (X)

—

—

- X: shared memory location
- P1's update on X should not be visible to P2 until P1 is guaranteed to complete.

Atomic Actions - Characteristics

An action is atomic

- If the process performing it is not aware of the existence of any other process,

&

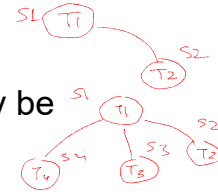
no other process is aware of the activity of the process during the time of atomic action

=> The process performing it does not communicate with other processes while the action is performed

=> The process performing it can detect no state changes except those performed by itself, & if it does not reveal its state changes until the action is complete

- For other processes it is invisible and instantaneous

- A **transaction** groups a sequence of actions
- Transaction may commit / abort
- In a distributed system a transaction may be executed at multiple sites
- Commit protocols
 - Enforce global atomicity property
 - Ensure that either all sites commit or all abort
- Difficulties in the design
 - **Generals Paradox**
 - Number of messages employed by a commit protocol be bounded by a fixed number of messages?



$O(n^m)$

Two-Phase Commit

- One process acts as **coordinator**
- Other processes are referred as **cohort**

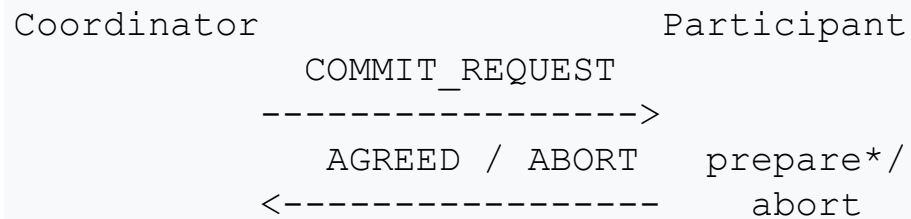
Phase 1:

Coordinator

- coordinator sends COMMIT_REQUEST to all cohorts
- waits for replies from all processes

Cohort

- on receiving a COMMIT_REQUEST:
 - if (local transaction is successful)
 - write Undo & redo logs in stable storage, and send an AGREED message to the coordinator
 - else send an ABORT



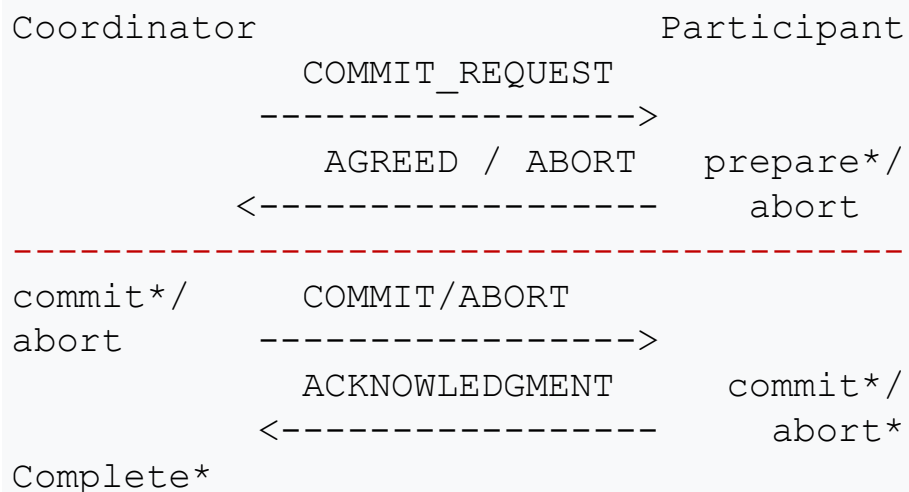
Phase 2:

Coordinator

- If (all cohorts reply AGREED)
 - writes COMMIT record into the log; sends COMMIT to all processes
- else (at least one process has replied ABORT)
 - sends ABORT to all.
- Coordinator then waits for ACK from all processes.
- If ACK is not received within timeout period, resend.
- If all ACKs are received, coordinator writes COMPLETE to log

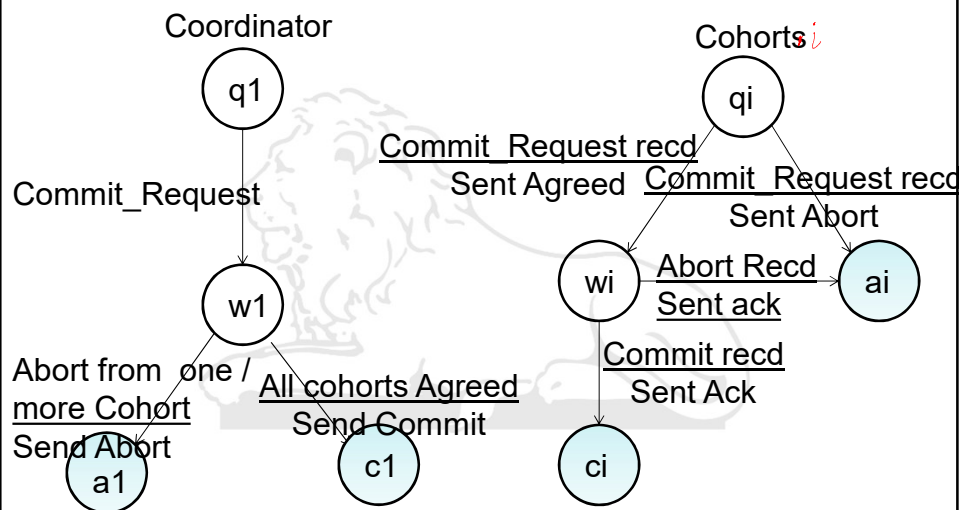
Phase 2:Cohort

- On receiving a COMMIT:
releases all resources/locks, and sends an ACK to coordinator
- On receiving an ABORT:
undoes the transaction using Undo log, releases all resources/locks, and sends an ACK



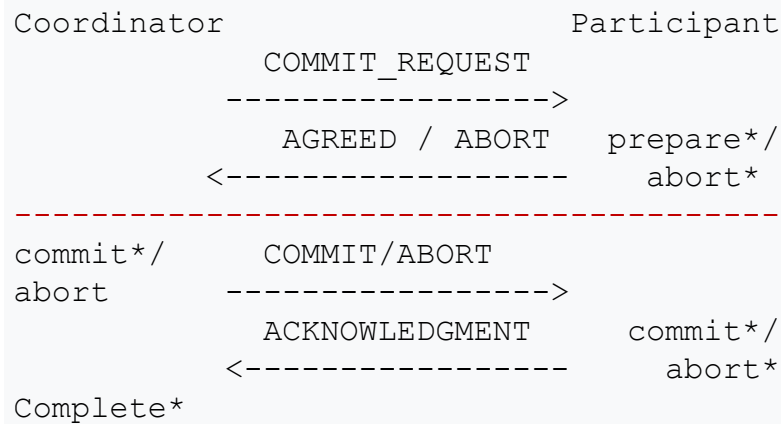
If there are no failures or message losses, it ensures global atomicity => either all processes commit or all of them aborts

Two Phase Commit – State Transition diagram



Site Failures

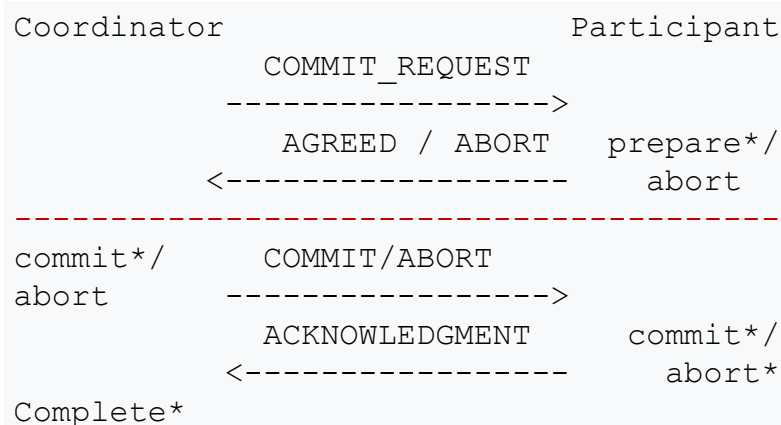
- Resilient to **crash failures**
 - Coordinator crashes before writing COMMIT



- On recovery coordinator broadcasts an ABORT

Site Failures

- Coordinator crashes after writing COMMIT but before writing COMPLETE



- On recovery coordinator broadcasts COMMIT

Site Failures

- Resilient to crash failures
 - A cohort crashes in phase I
 - Coordinator aborts transaction as it did not receive reply from the crashed cohort
 - A cohort crashes in phase II after writing its UNDO and REDO logs and after sending AGREED / ABORT
 - On recovery check with the coordinator about the decision

Two-phase protocol

- **is blocking** – crash of coordinator can block all processes (cohorts). After a participant has sent an agreement message, it will block until a commit or ^{abort}rollback is received.
- If a network partition separates coordinator from cohorts?
- Designed to handle fail-stop faults only
- The two-phase commit protocol cannot recover from a failure of both the coordinator and a cohort member during the commit phase.